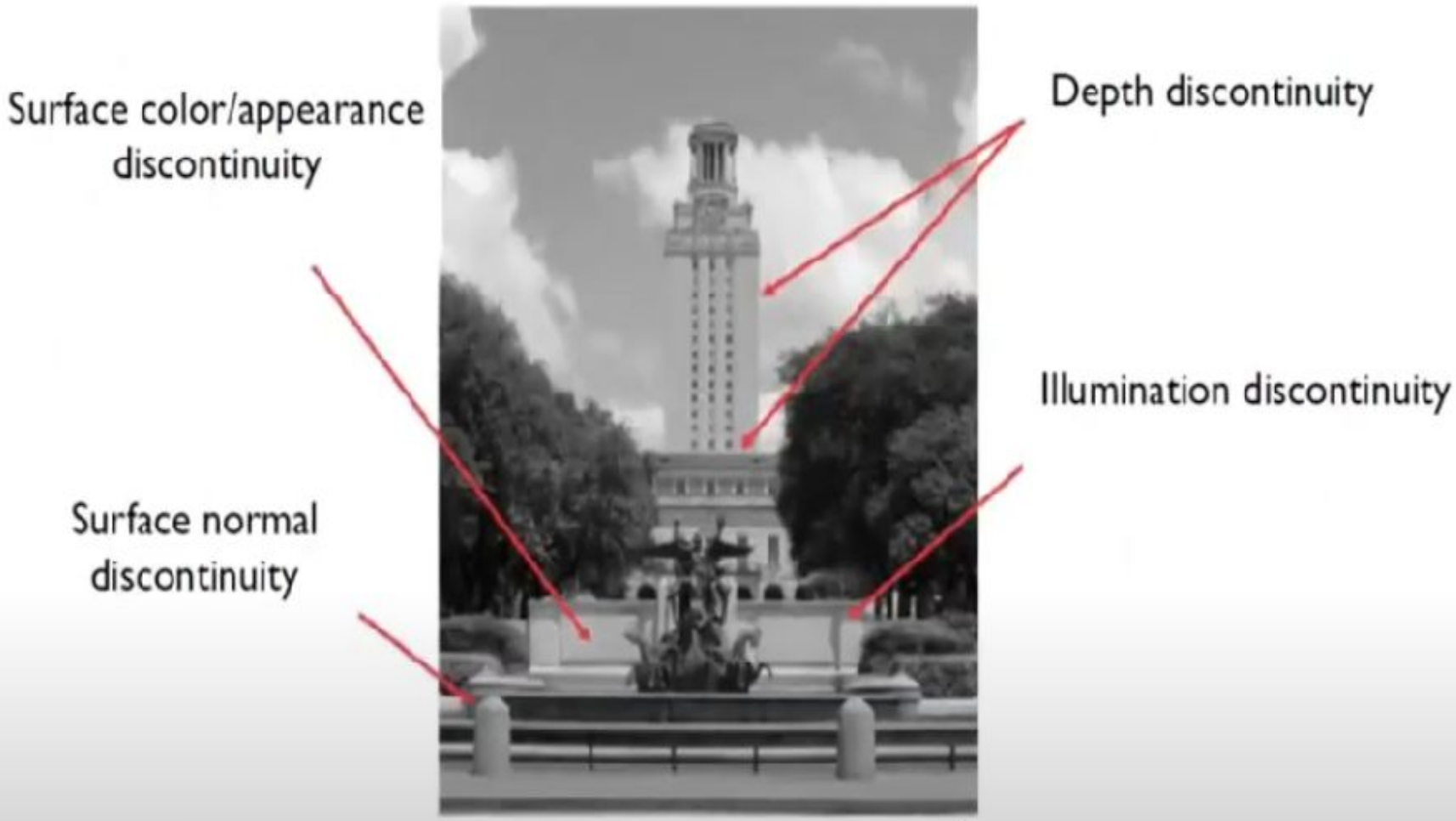
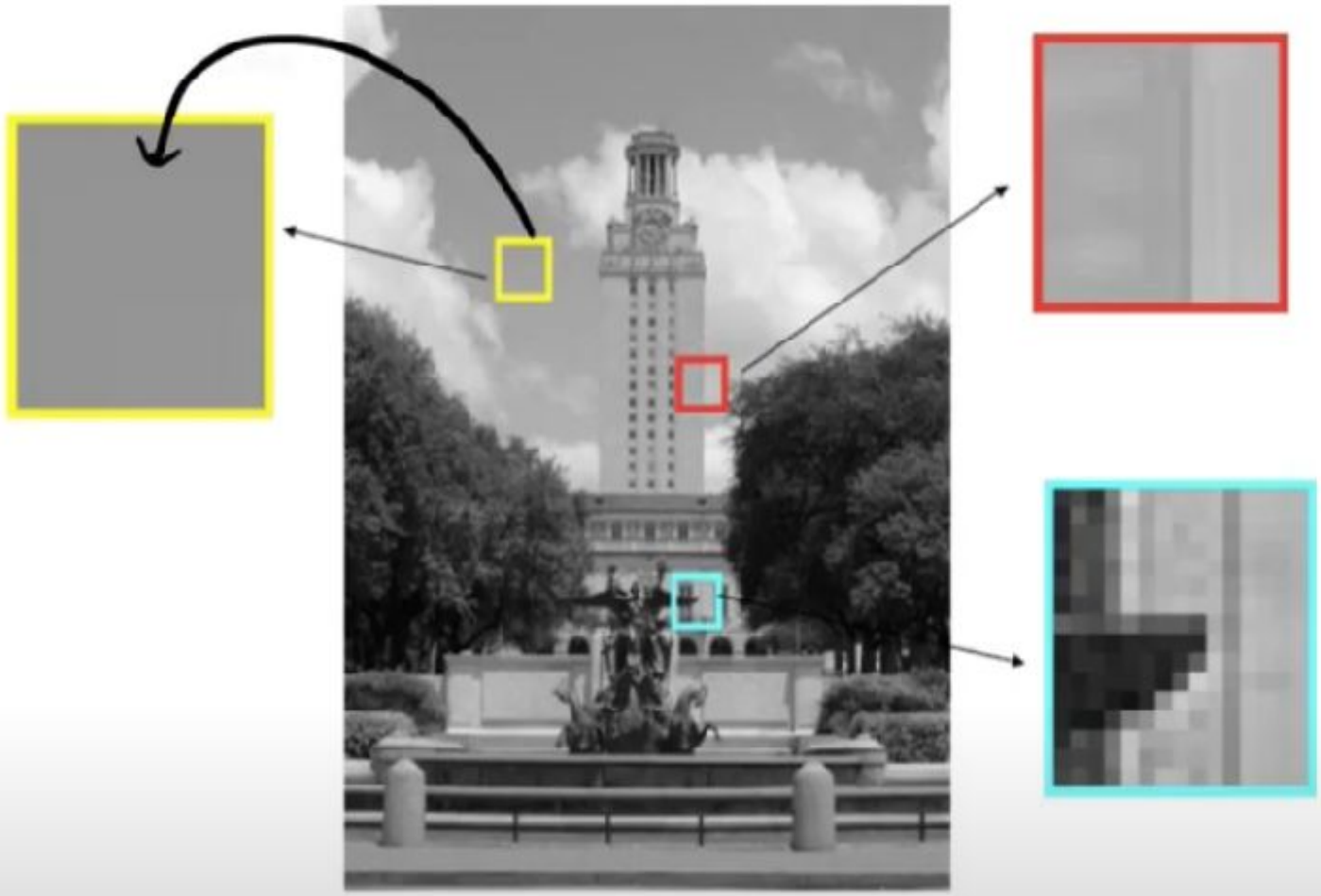
Edge Detection

How are Edges Caused?

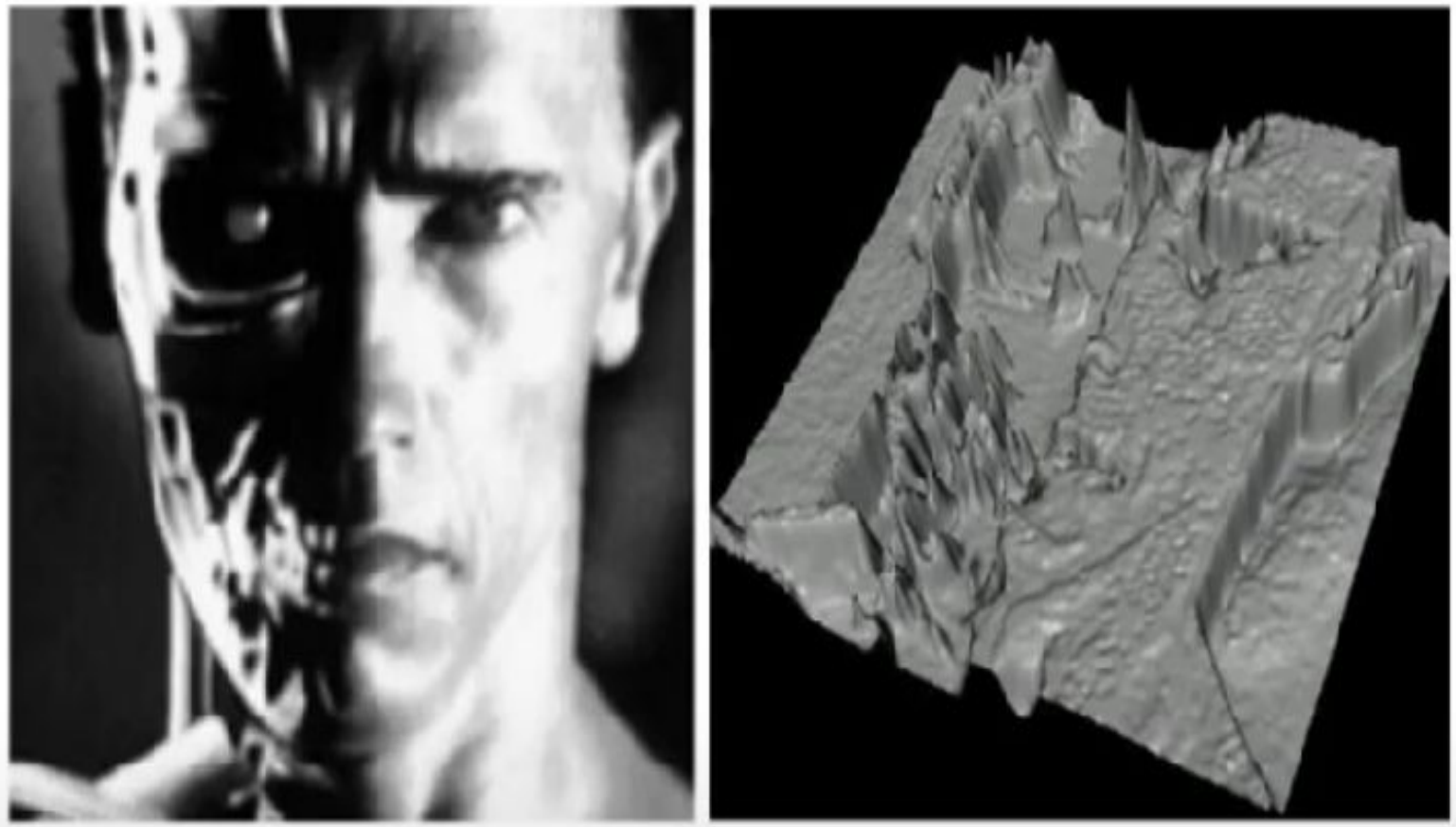
- Variety of factors:



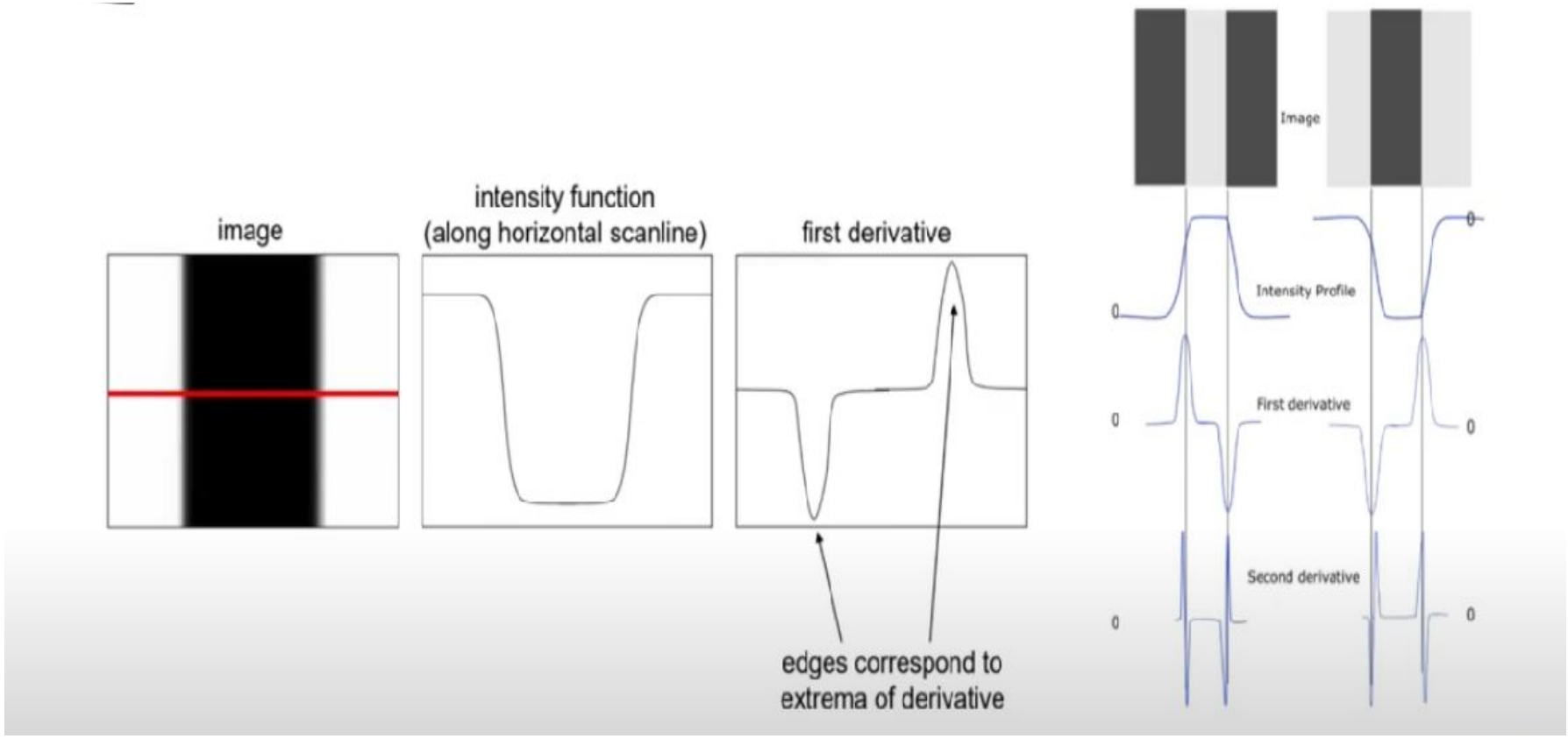
Looking More Locally



- Edges look like steep cliffs



Derivatives and Edges



Derivatives with convolution

- For 2D function, $f(x, y)$, the partial derivative is:

$$\frac{\partial f(x,y)}{\partial x} = \lim_{\varepsilon \to 0} \frac{f(x+\varepsilon, y) - f(x,y)}{\varepsilon}$$

- For discrete data, we can approximate using finite differences:

$$\frac{\partial f(x,y)}{\partial x} \approx \frac{f(x+1, y) - f(x,y)}{1}$$



$$\frac{\partial f(x,y)}{\partial x}$$



-1	1

$$\frac{\partial f(x,y)}{\partial y}$$

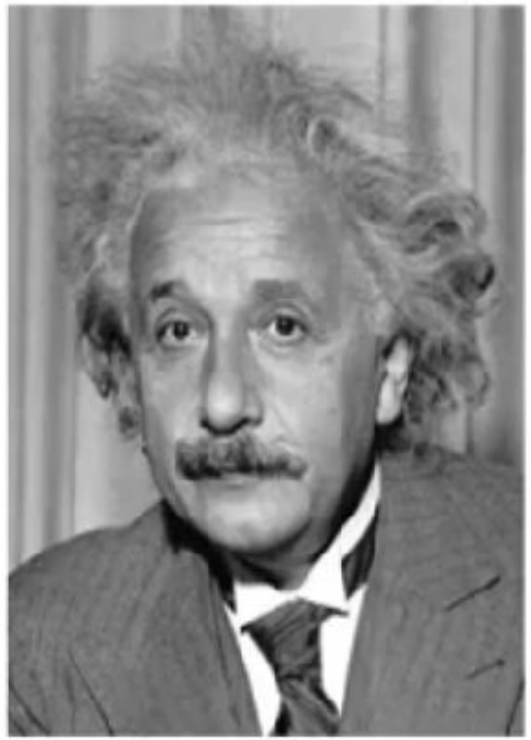


-1
1

or

1
-1

Sobel Edge Detection Filters

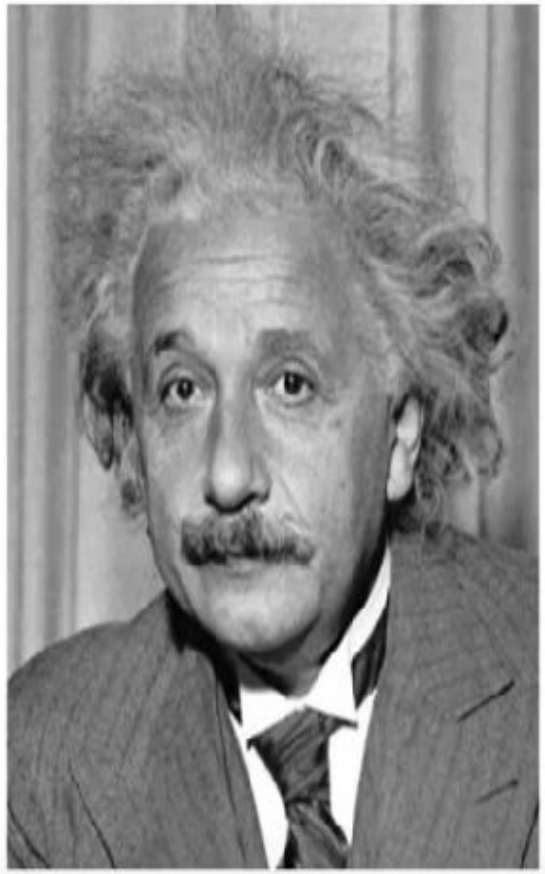


1	0	-1
2	0	-2
1	0	-1

Sobel



Vertical Edge
(absolute value)



1	2	1
0	0	0
-1	-2	-1

Sobel

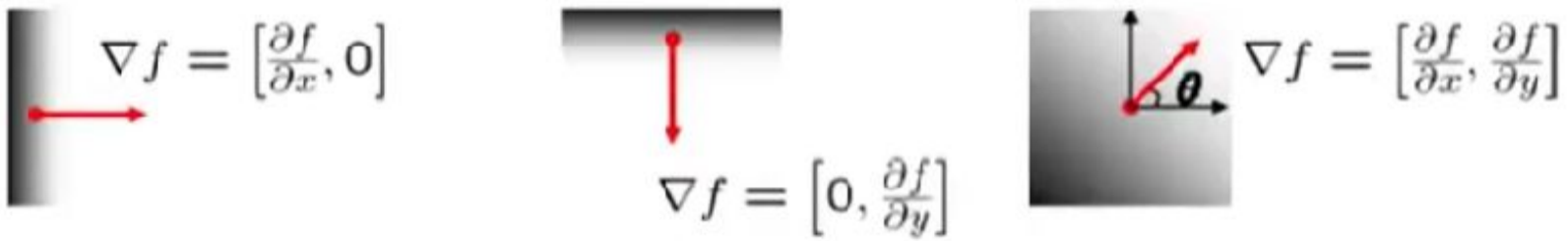


Horizontal Edge
(absolute value)

	M_x	M_y
Prewitt	$\begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$	$\begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$
Sobel	$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$	$\begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$
Roberts	$\begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$	$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$

Image Gradients

- The gradient of an image $\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}\right]$
- The gradient points in the direction of most rapid change in intensity



- The **gradient direction** (orientation of edge normal) is given by:

$$\theta = \tan^{-1}\left(\frac{\partial f}{\partial y} / \frac{\partial f}{\partial x}\right)$$

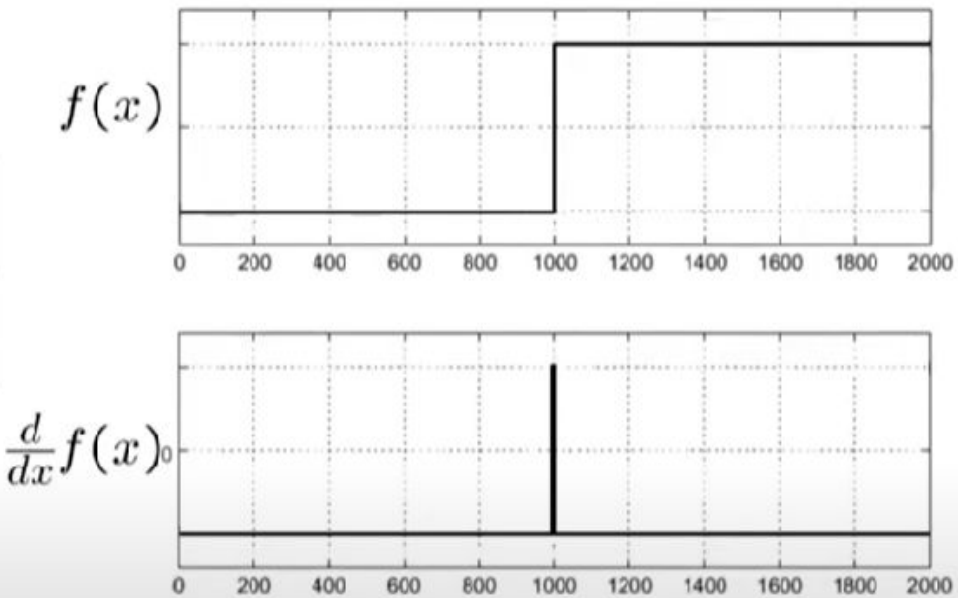
- The **edge strength** is given by the magnitude $\|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$

Derivative with No Noise

- Consider a single row or column of the image
  - Plotting intensity as a function of position gives a signal



Input image with no noise

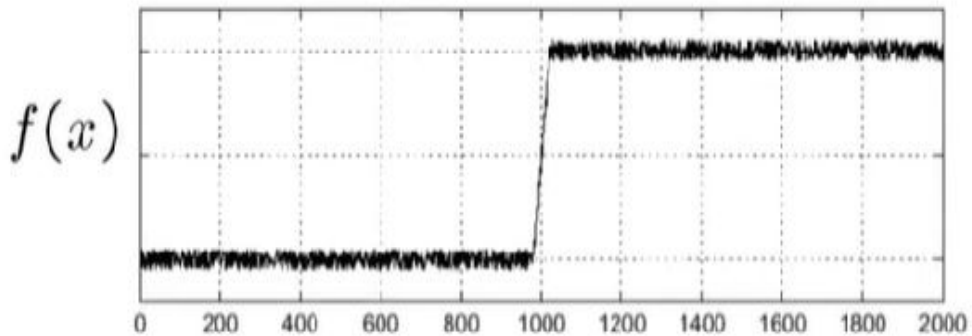


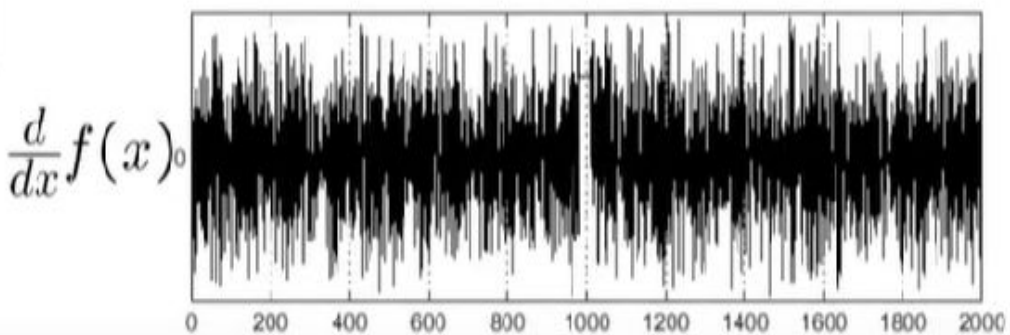
Where is the edge?

Effect of Noise



Noisy input image

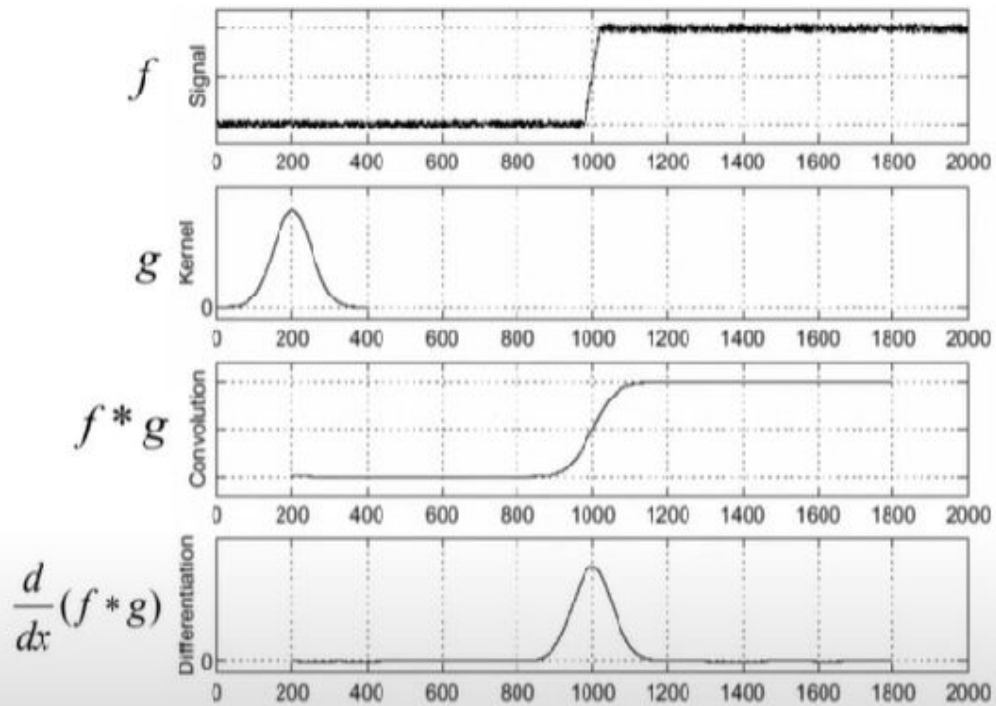




Now, where is the edge?

Effect of Noise

- Smooth first, and look for peaks in $\frac{d}{dx}(f * g)$

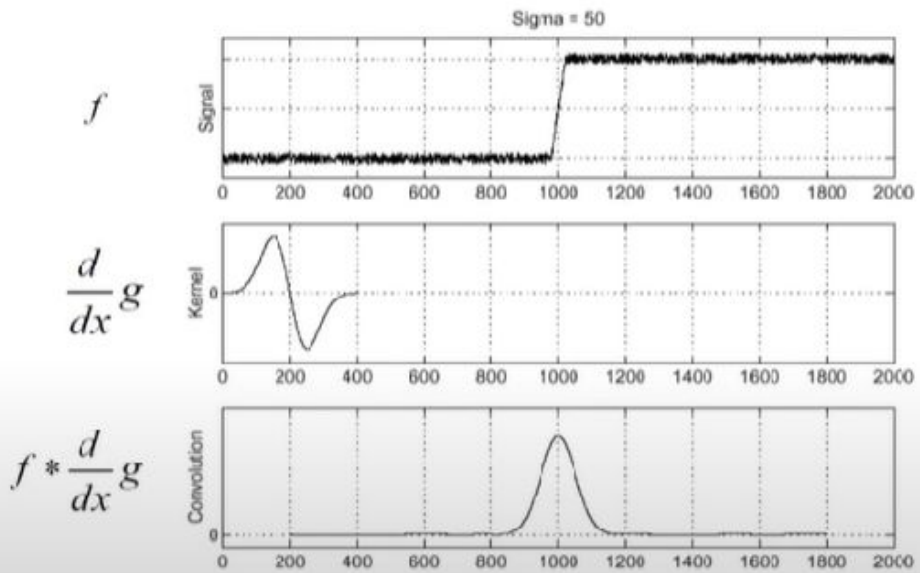


Derivative theorem of Convolution

- Differentiation is achieved through convolution, and convolution is associative:

$$\frac{d}{dx}(f * g) = f * \frac{d}{dx}g = \frac{d}{dx}f * g$$

- This saves us an operation:



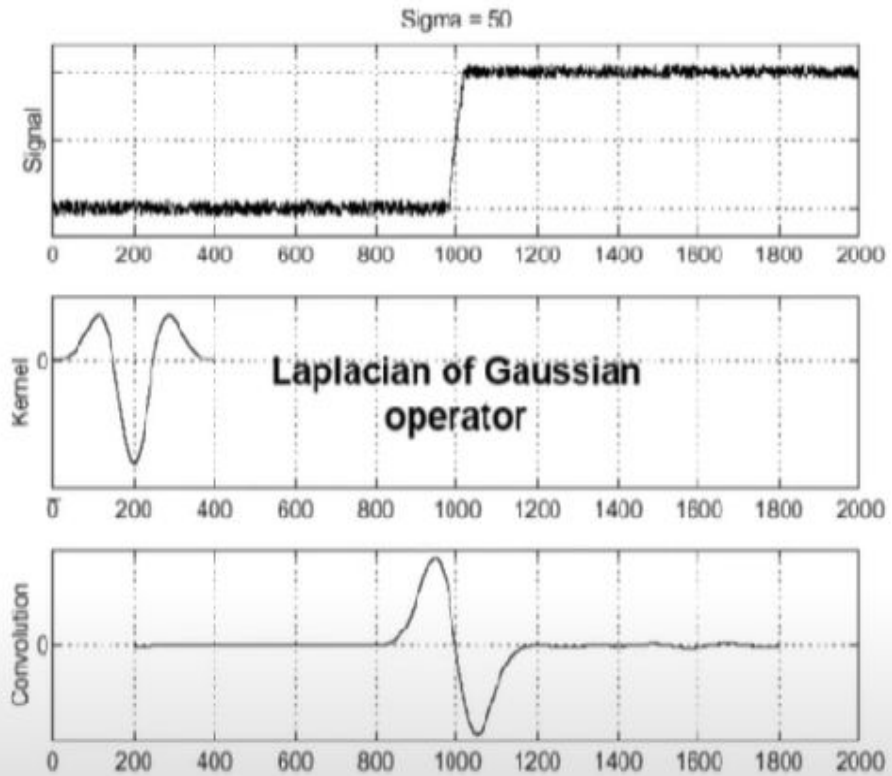
What about Second Derivative?

- Edge by detecting **zero-crossing** of bottom graph

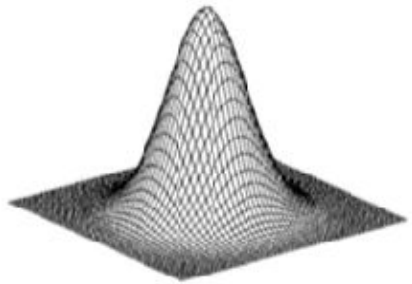
f

$\frac{\partial^2}{\partial x^2} h$

$(\frac{\partial^2}{\partial x^2} h) \star f$

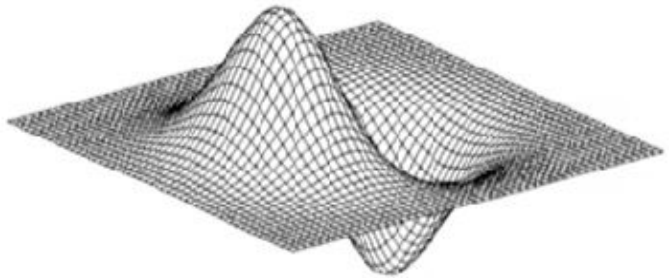


Derivative and Laplacian of Gaussians



Gaussian

$h_\sigma(x,y) = \frac{1}{2\pi\sigma^2} \exp^{-\frac{u^2+v^2}{2\sigma^2}}$



Derivative of Gaussian (x)

$\frac{\partial}{\partial x} h_\sigma(u,v)$

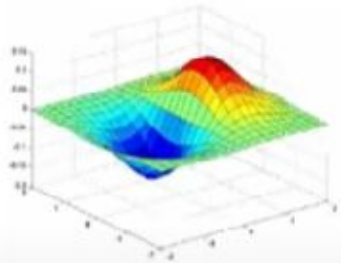


Laplacian of Gaussian

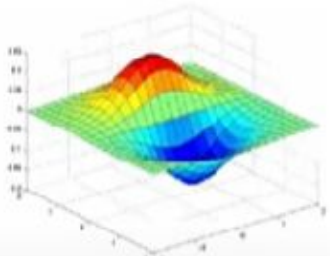
$\nabla^2 h_\sigma(u,v)$

with ∇^2 the Laplacian operator

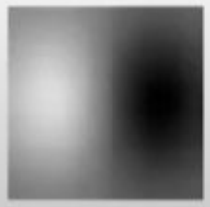
$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$



x-direction



y-direction





Which one finds horizontal/vertical edges?

Source: S Seitz, R Urtasun

Compute of Gradients





X-Derivative of Gaussian



Y-Derivative of Gaussian



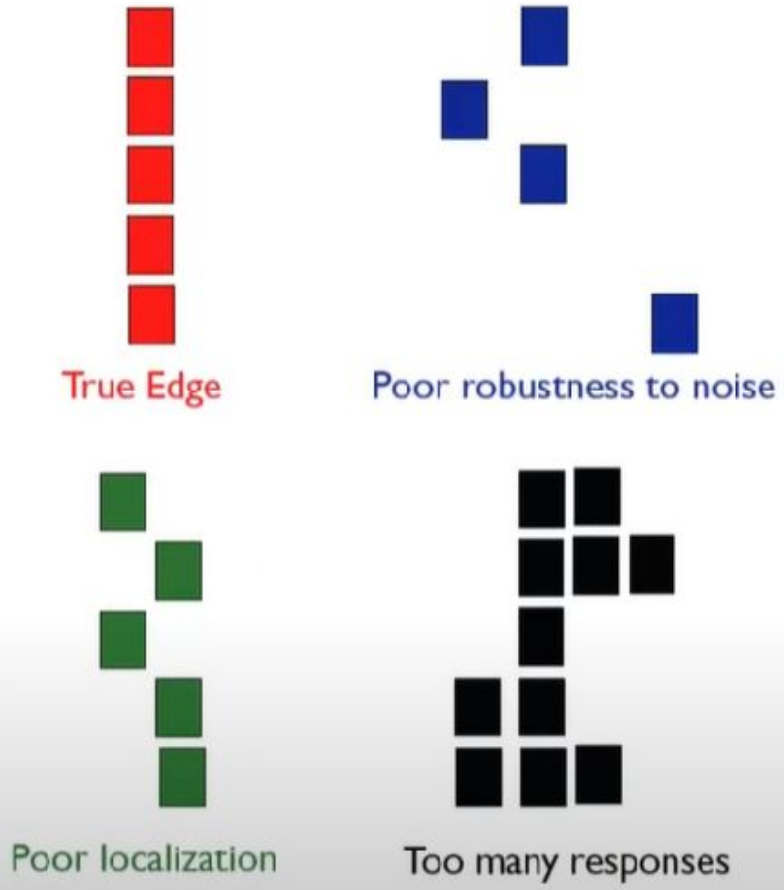
Gradient Magnitude

Properties of an Edge Detector



Properties of an Edge Detector

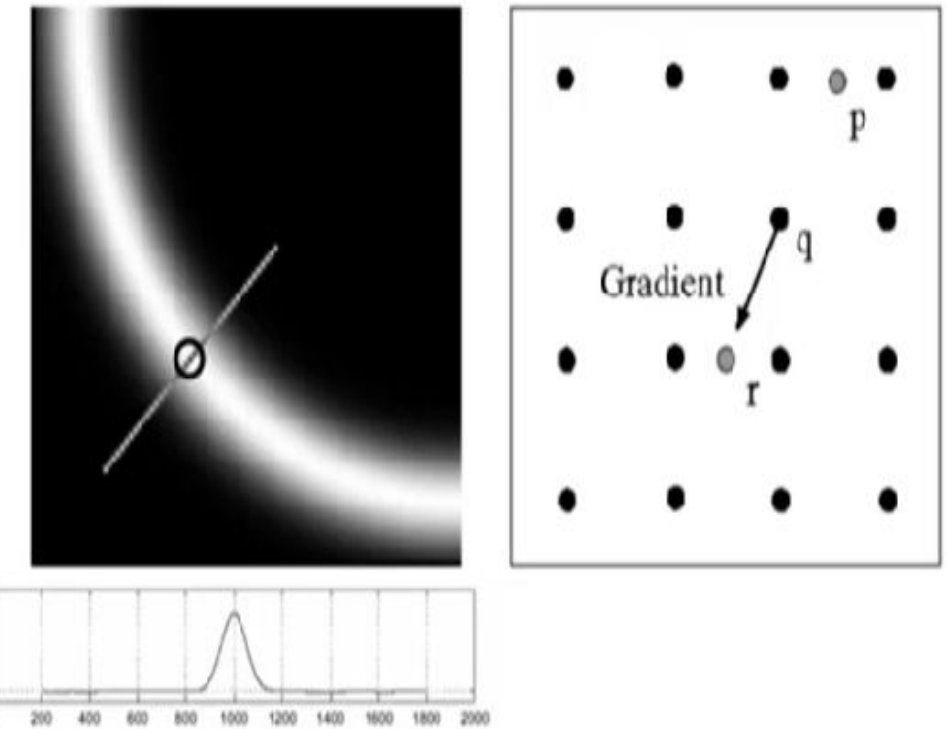
- Criteria for a good edge detector?
  - **Good detection:** find all real edges, ignoring noise or other artifacts
  - **Good localization:** detect edges as close as possible to true edges
  - **Single response:** return one point only for each true edge point



Non-Maxima Suppression



where is the edge?



- Check if pixel is local maximum along gradient direction:
  - Could require checking interpolated pixels p and r

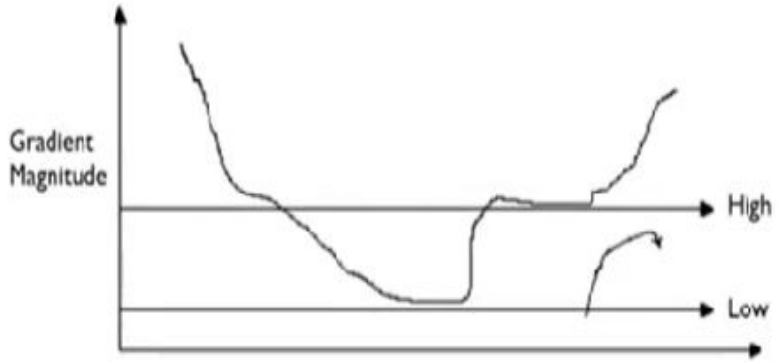
Non-Maxima Suppression



Before and after non-maxima suppression

Hysteresis Thresholding

- Check for well-connected edges. How?
  - Use **hysteresis**: use a *high* threshold to start edge curves and a *low* threshold to continue them.



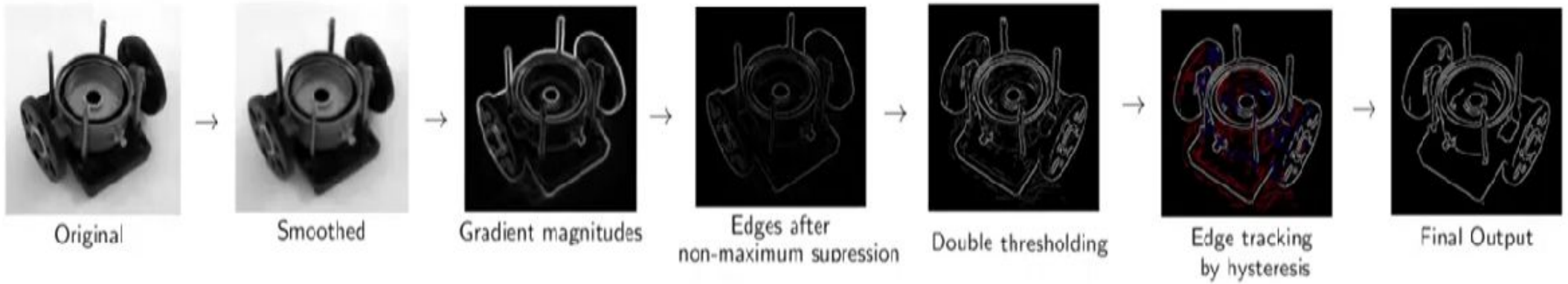
- How does it work?
  - If gradient at pixel > 'High' $\Longrightarrow$ **'edge pixel'**
  - If gradient at pixel < 'Low' $\Longrightarrow$ **'non-edge pixel'**
  - If gradient at pixel $\geq$ 'Low' and $\leq$ 'High' $\Longrightarrow$ **'edge pixel'** iff it is connected to an 'edge pixel' directly or via pixels between 'Low' and 'High'



Canny Edge Detector

- Probably the most widely used edge detector in computer vision (J Canny 1986 [1])
- Algorithm:
  1. Filter image with derivative of Gaussian
  2. Find magnitude and orientation of gradient
  3. Non-maximum suppression
  4. Linking and thresholding (hysteresis):
     - Define two thresholds: low and high
     - Use the high threshold to start edge curves and the low threshold to continue them

Canny Edge Pipeline and Examples





Canny Edges

Effect of σ in Canny Edge Detector

- The choice of σ (Gaussian kernel spread/size) depends on desired behavior
  - large σ detects large-scale edges
  - small σ detects fine edges

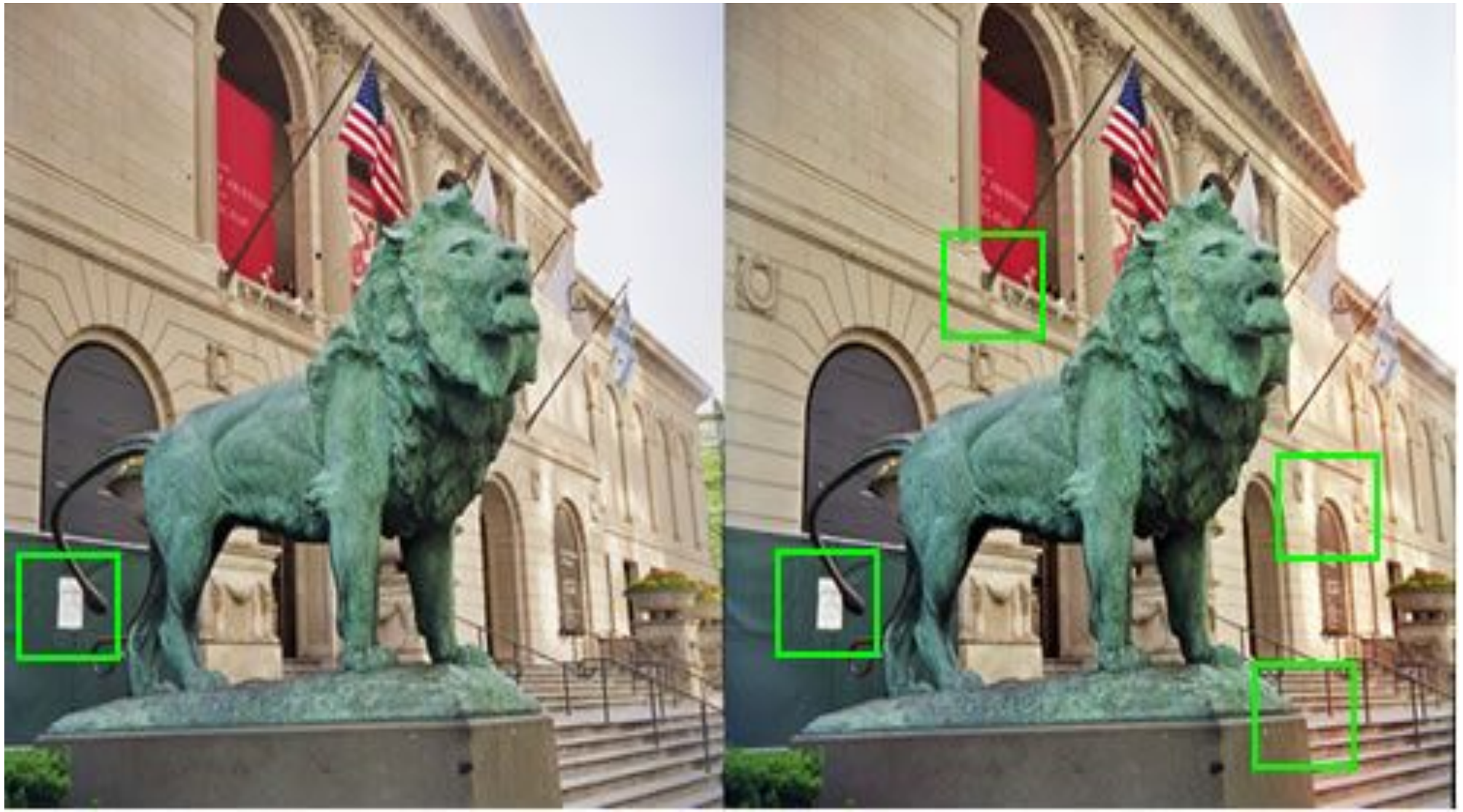


original | Canny with $\sigma = 1$ | Canny with $\sigma = 2$

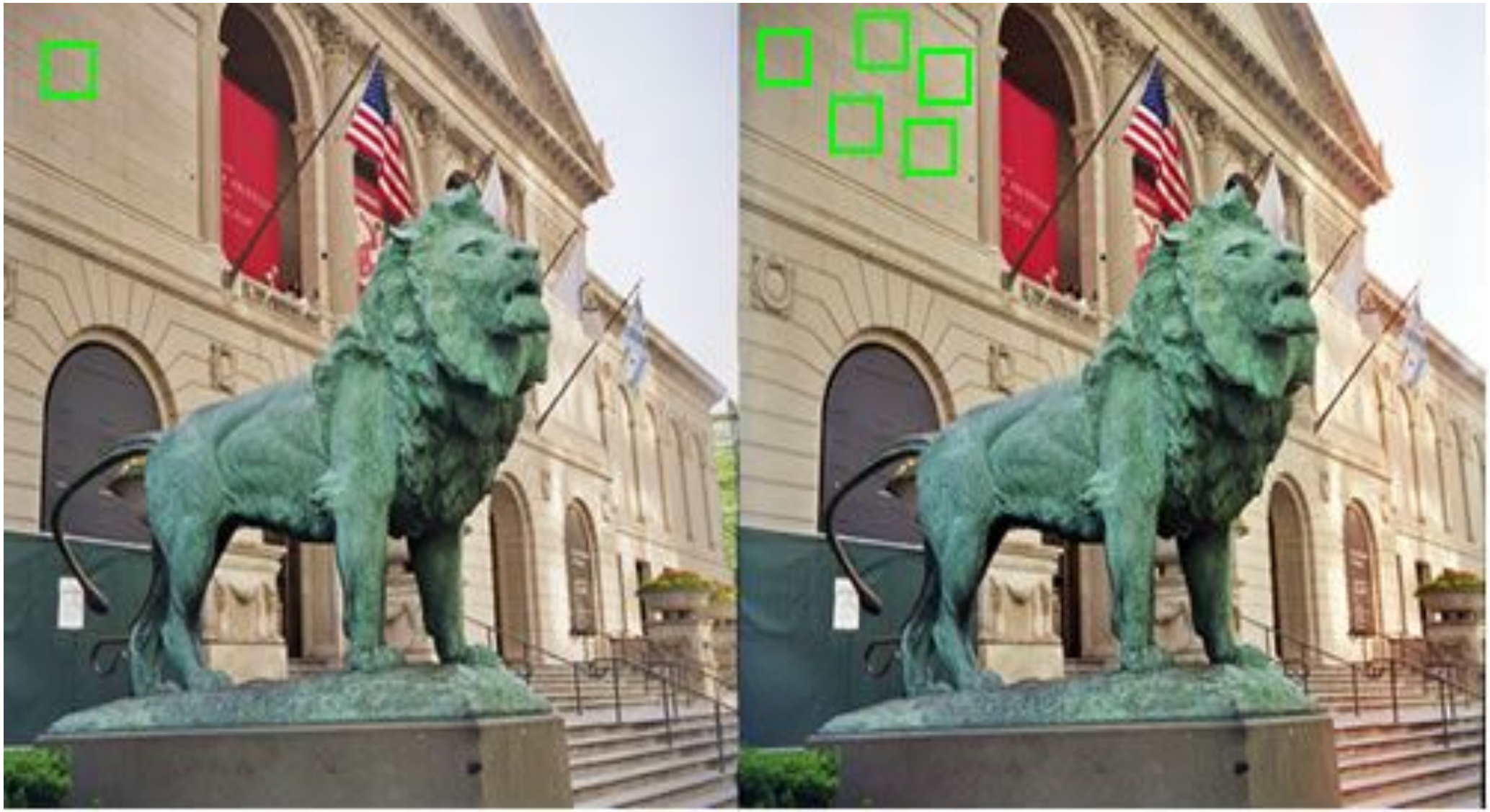
What are Features?

- Several computer vision tasks require finding matching points across several frames or views.
- With that info, you could really do a lot of stuff.
- An example. When doing stereo imaging, you want to know a few corresponding points between the two views.
- Once you do, you can triangulate almost all points on the image
- We can accomplish this with something called *features*.

Patches

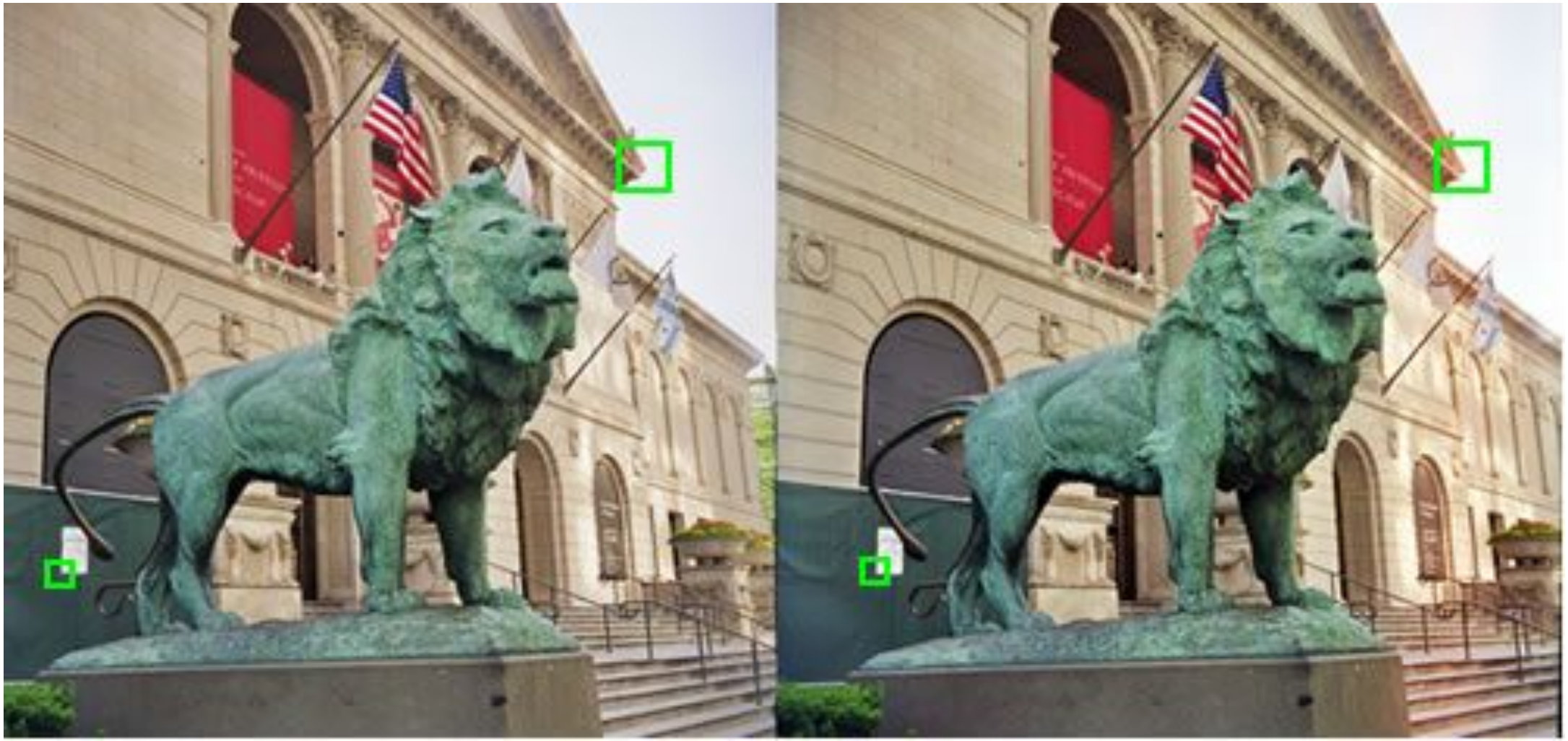


- You want to find the left image's green path in the right.
- And you can do that quite easily.
- The white thingy makes the patch quite unique.
- Even something as trivial as template matching would be able to find it.
- But, not all patches are so uniquely recognizable



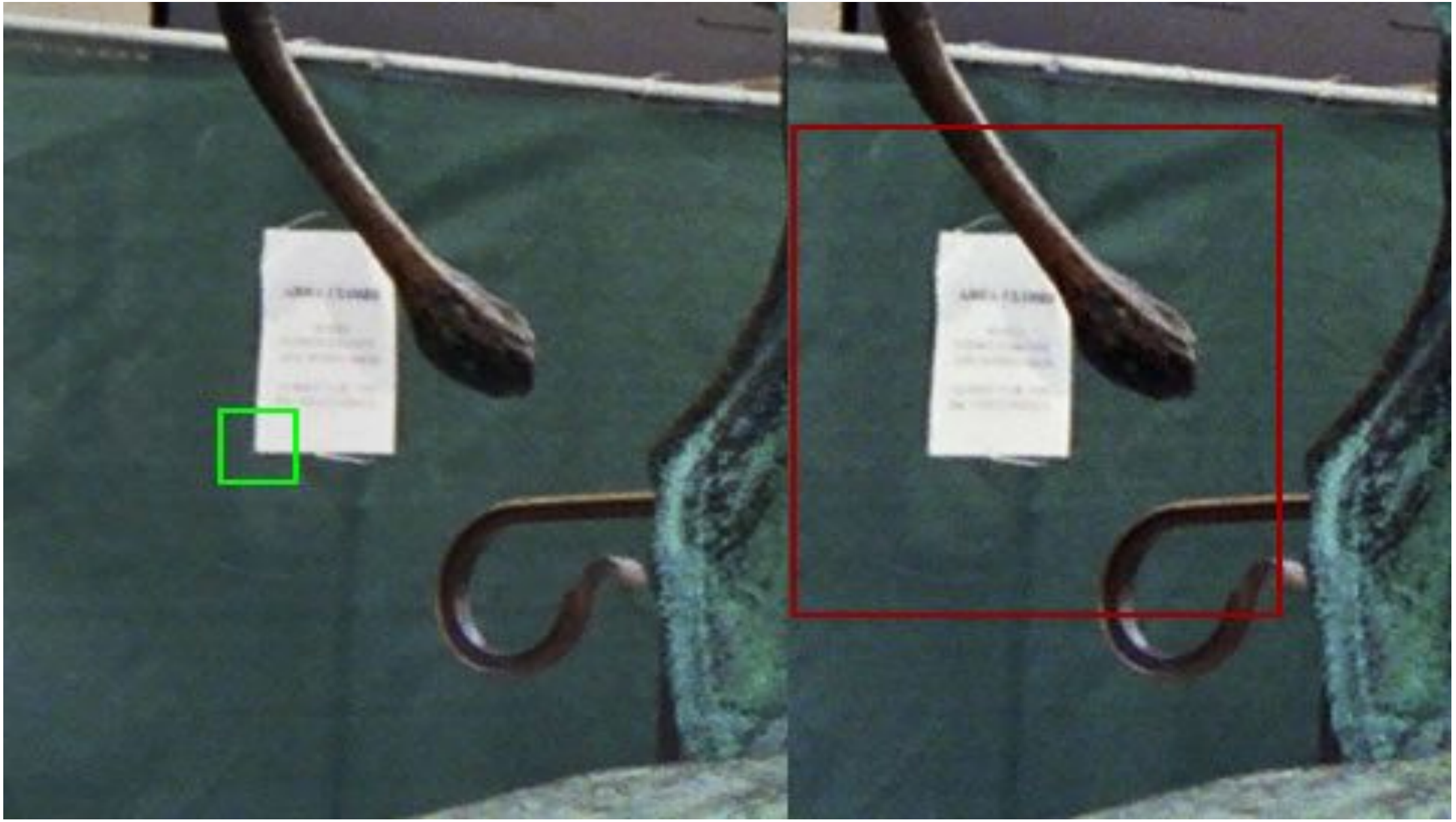
- There are no unique "features" to identify on the wall.
- So you'll have problem finding corresponding points.
- So the patches approach isn't that great.

Corners - Corners in an image seem to be perfect for such tracking tasks.



These corners are perfect! Why?

- Uniquely identifiable
- These points are uniquely identifiable.
- What do I mean by that? Here's what.
- Lets say you're trying to find the green corner in the right image
- You know that it'll be somewhere around the same location.
- So you can narrow down the "search region".
- And within this search region, there would be only one point that resembles the corner.
- Of course, the assumption here is that there isn't a massive difference between the two images. And this is usually a reasonable assumption.



- Stable
- These points usually don't keep moving around in the image.
- This helps tracking.
- And any motion of this point, even a little one, produces a large variation.
- You can clearly see the point moving around.

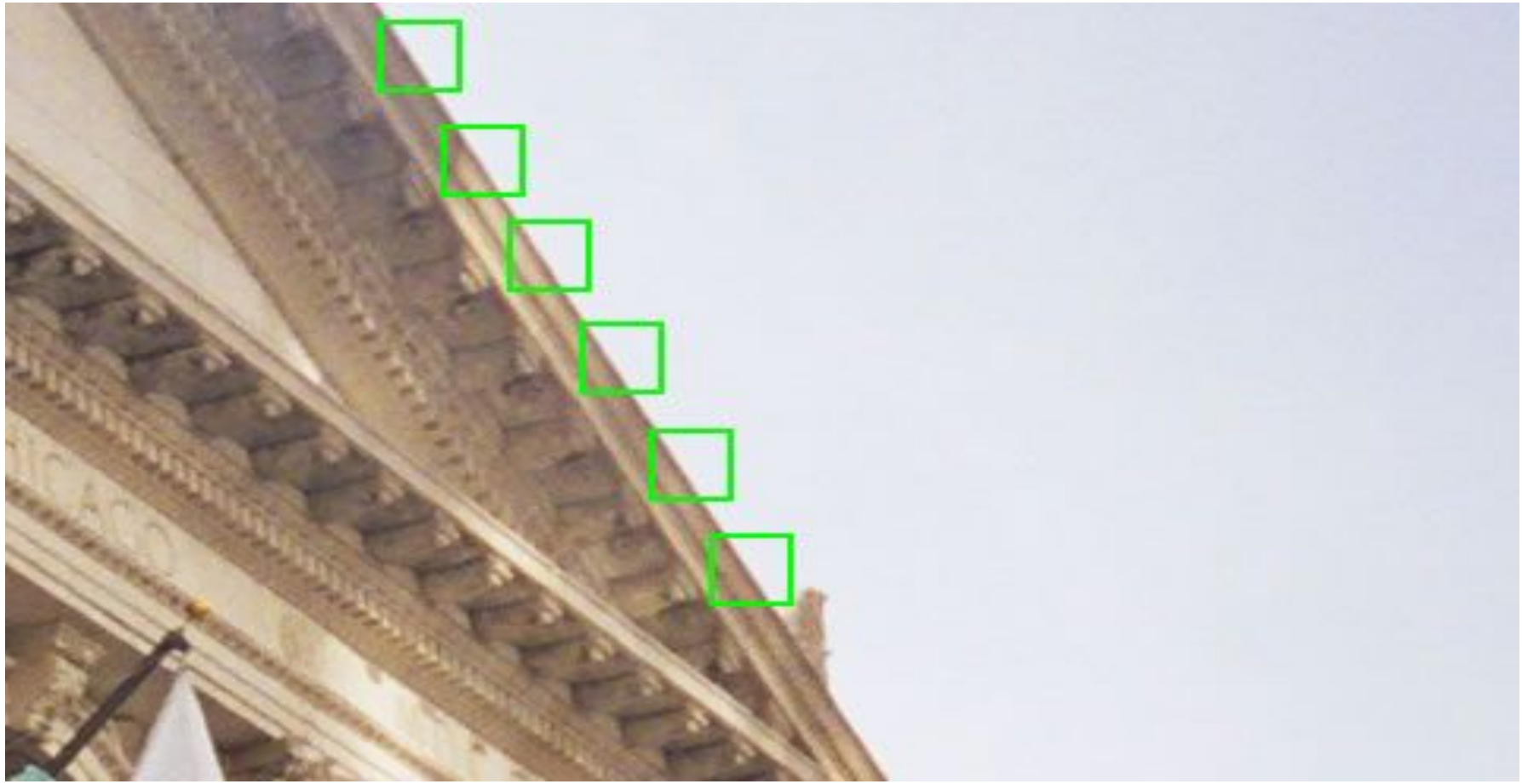
A bad feature

- I'll try to make the idea of a "corner" more concrete.
- We'll use some math to do this.
- How do you identify a bad feature?
- Something that doesn't have a lot of variation... like the example of the wall above.
- There are no edges or corners in the feature.
- So, the first derivative is flat, in both directions, x and y.
- Or, the first derivative is flat in all directions (all directions are a certain combination of the x and y component).
- So, the second derivative also does not change in any direction.

An edge

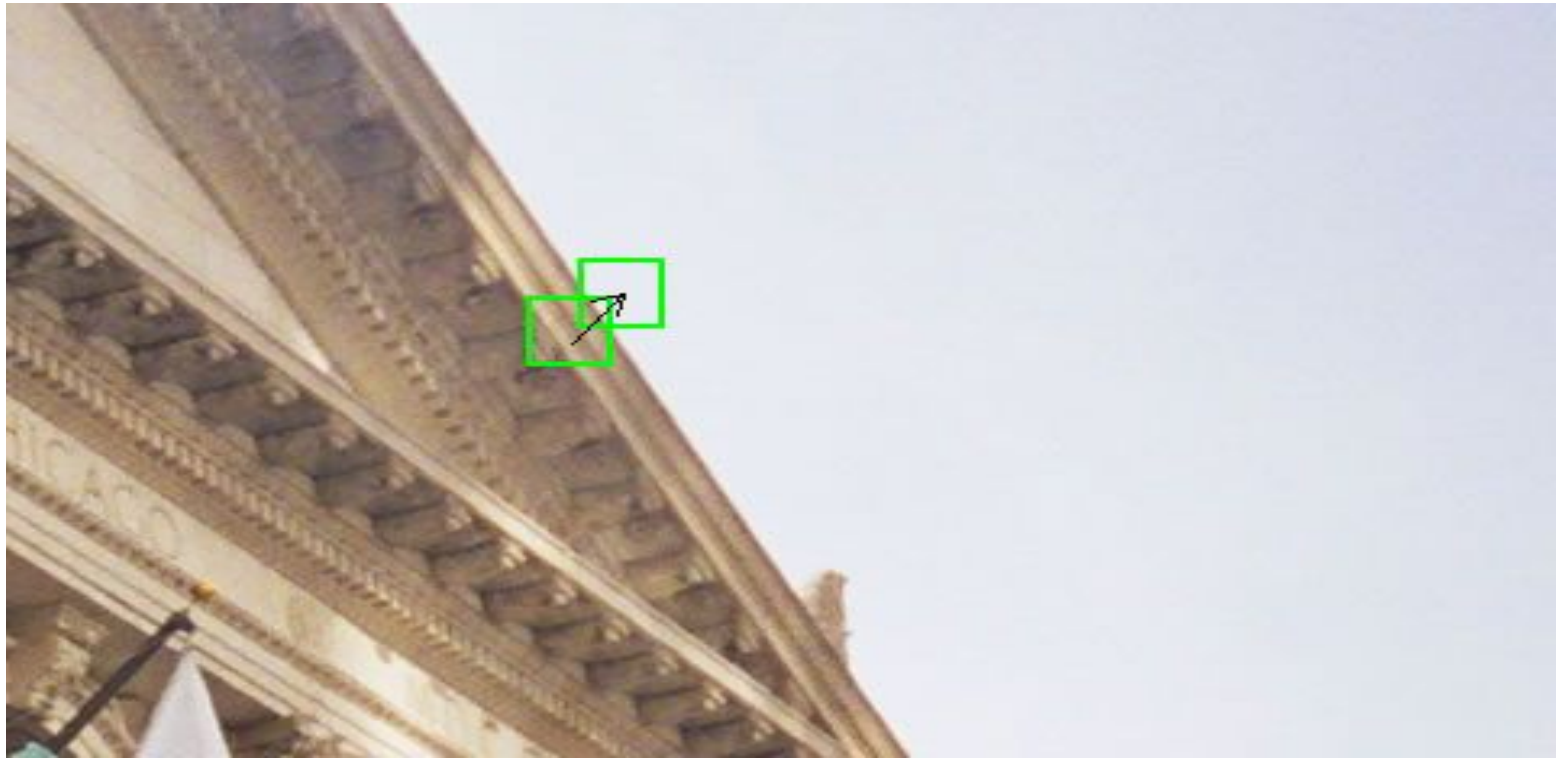
- An edge is a bad feature as well.
- If you move in the direction of the edge, you won't even know you're moving.
- For example, if you move along the edge at the top of the building and the sky, you won't even realize it.

Motion along edge



Motion Perpendicular to an edge

- But if you move from the building to the sky (perpendicular to the edge) you'll "see" motion. This is the only direction where you can accurately tell how fast the object is moving.



- So edges aren't that useful as features.
- The first derivative changes in only the one direction (perpendicular to the edge).
- Some progress, but not that good.
- So, the second derivative also changes in only one direction.

A Corner

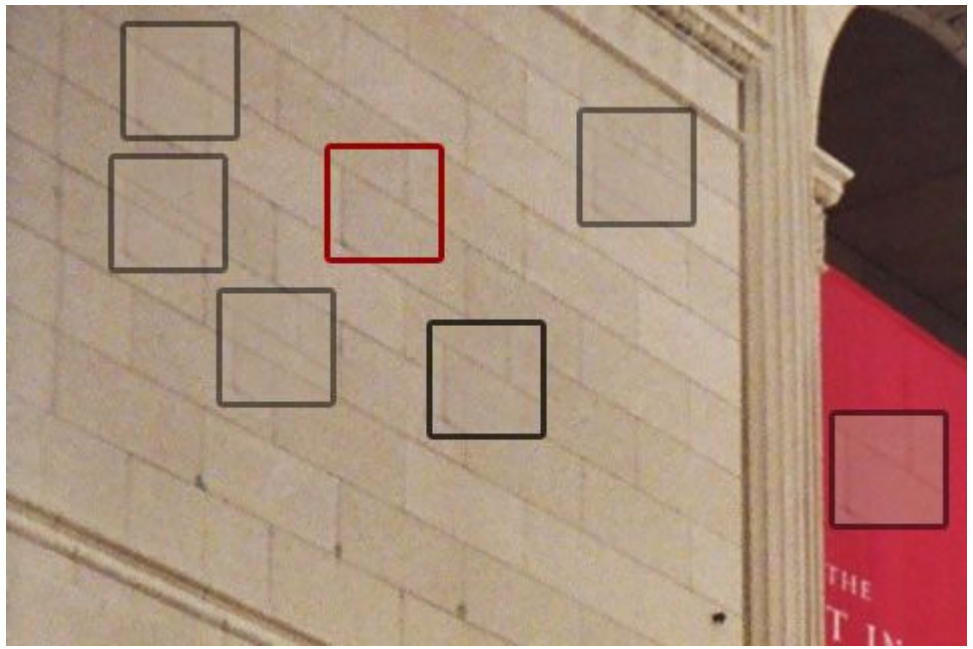
- A corner is an awesome feature!
- There's variation all around a corner.
- So, the derivative changes in all directions.
- So the second derivative also changes in all directions! Great! And
- We can write pretty efficient programs to calculate that at all points.
- If the first derivative keeps changing around a point, you know you have a corner.
- And you also know you have a good feature to track!

So what exactly is a feature?

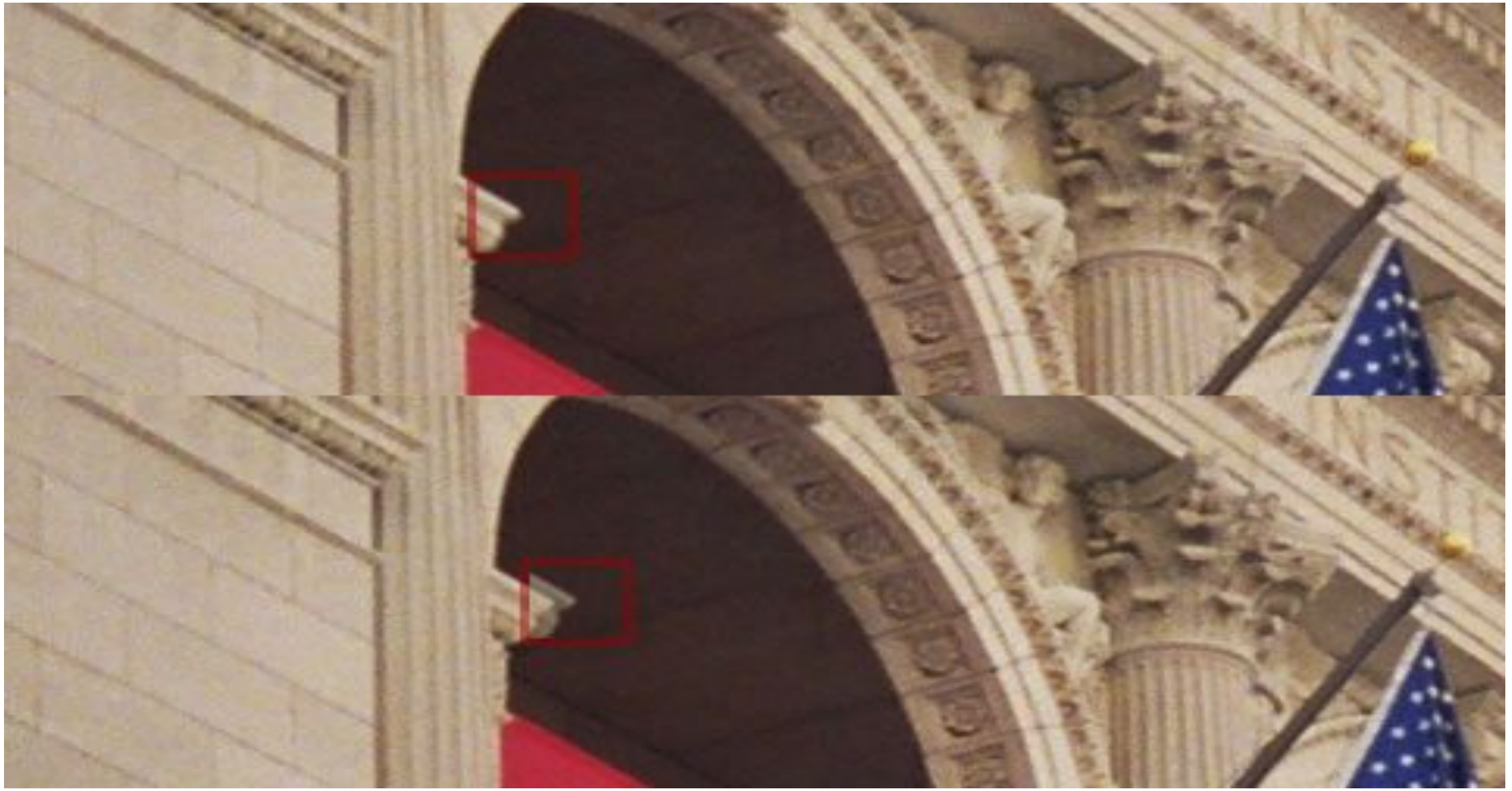
- Until now, we've talked about patches of images that are uniquely identifiable.
- But, exactly *how* do you identify them?
- One way is to directly use the pixel RGB values - and this is exactly what happens when using template matching.
- You represent the patch as a sequence of numbers and compute the squared difference.
- However, you could get more sophisticated and use some other numbers to represent the patches (the three dominant colors, metadata like the GPS coordinates, etc).
- In computer vision, this is termed as a *feature descriptor* and there exist several techniques to *describe* an image patch using a sequence of numbers.

- The Harris Corner Detector is a mathematical operator that finds features in an image.
- It is simple to compute, and is fast enough to work on computers.
- Also, it is popular because it is rotation, scale and illumination variation independent.
- However, the Shi-Tomasi corner detector, the one implemented in OpenCV, is an improvement of this corner detector

- Our aim is to find little patches of image (or "windows") that generate a large variation when moved around. Have a look at this image:



- The red square is the window we've chosen.
- Moving it around doesn't show much of variation.
- That is, the difference between the window, and the original image below it is very low.
- So you can't really tell if the window "belongs" to that position.
- Of course, if you move the window too much, like onto the reddish region, you're bound to see a big difference.
- But we've moved the window too much. Then its Not good.



- Even the little movement of the window produces a noticeable difference.
- This is the kind of window we're looking for.
- Here's how it translates mathematically:
-

$$E(u,v) = \sum_{x,y} w(x,y)[I(x+u,y+v) - I(x,y)]^2$$

- E is the difference between the original and the moved window.
- u is the window's displacement in the x direction
- v is the window's displacement in the y direction
- w(x, y) is the window at position (x, y). This acts like a mask. Ensuring that only the desired window is used.
- I is the intensity of the image at a position (x, y)
- I(x+u, y+v) is the intensity of the moved window
- I(x, y) is the intensity of the original

- We've looking for windows that produce a large E value.
- To do that, we need to high values of the terms inside the square brackets.
- So, we maximize this term:
-

$$\sum_{x,y} [I(x+u, y+v) - I(x,y)]^2$$

- Then, we expand this term using the Taylor series.
- It's just a way of rewriting this term in using its derivatives.

$$E(u,v) \approx \sum_{x,y} [I(x,y) + uI_x + vI_y - I(x,y)]^2$$

- See how the I(x+u, y+v) changed into a totally different form ($I(x,y)+uI_x + vI_y$)?
- Thats the Taylor series in action.
- And because the Taylor series is infinite, we've ignored all terms after the first three. It gives a pretty good approximation. But it isn't the actual value.

- Next, we expand the square. The I(x,y) cancels out, so its just two terms we need to square. It looks like this:

$$E(u,v) \approx \sum_{x,y} u^2 {I_x}^2 + 2uv I_x I_y + v^2 {I_y}^2$$

- Now this messy equation can be tucked up into a neat little matrix form like this:

$$E(u,v) \approx \begin{bmatrix} u & v \end{bmatrix} \left(\sum \begin{bmatrix} I_x{}^2 & I_x I_y \\ I_x I_y & I_y{}^2 \end{bmatrix} \right) \begin{bmatrix} u \\ v \end{bmatrix}$$

- Now, we rename the summed-matrix, and put it to be M:

$$M = \sum w(x,y) \begin{bmatrix} I_x{}^2 & I_x I_y \\ I_x I_y & I_y{}^2 \end{bmatrix}$$

$$E(u,v) \approx \begin{bmatrix} u & v \end{bmatrix} M \begin{bmatrix} u \\ v \end{bmatrix}$$

- It was figured out that eigenvalues of the matrix can help determine the suitability of a window.
- A score, R, is calculated for each window:
- All windows that have a score R greater than a certain value are corners. They are good tracking points.

$$R = \det M - k(trace\, M)^2$$

$$\det M = \lambda_1 \lambda_2$$

$$trace\, M = \lambda_1 + \lambda_2$$

- The Harris Corner Detector is just a mathematical way of determining which windows produce large variations when moved in any direction.
- With each window, a score R is associated.
- Based on this score, you can figure out which ones are corners and which ones are not.

Hough Transform

- **Hough transform** is a feature extraction method used in image analysis.
- Hough transform can be used to isolate features of any regular curve like lines, circles, ellipses, etc.
- Hough transform in its simplest from can be used to detect straight lines in an image.
- A generalized Hough transform can be used in applications where simple analytic description of features is not possible.
- Due to the computational complexity of the algorithm, people generally refrain from using it.
- Moreover learning based methods can extract complex features from the image or a video that better suit the problem we are trying to solve.

- A straight line is the simplest boundary we can recognize in an image. Multiple straight lines can form a much complex boundary.
- We transform the image space into hough space. By doing this we convert a line in image space to a point on hough space.

